Prerequisites and objectives

- Prerequisites are pointers, object lifetimes, Stack and Queue ADTs, and Chapter 1 invariants.
- Distinguish a node's position, value, and ownership. Identify operations that actually need a search.
- Trace singly linked, circular, and doubly linked lists, linked stacks and queues, and polynomial addition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Valid singly linked states

- Each node contains a value and next. Head points to the first node. Null represents an empty list.
- Following next from head forms a finite acyclic chain ending in null. Exactly one container is responsible for releasing each node.
- If tail is stored, it points to the last node in a nonempty list and `tail.next=null`. Both head and tail are null for an empty list.
- External pointers may be nonowning references. Deleting the referenced node invalidates them. An address does not imply a live object at that address.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Insertion with a known predecessor

To insert new node x after p:

1. Allocate and construct x, setting `x.next=p.next`.
2. Set `p.next=x`. If p was the tail, set `tail=x`.
3. Update the element count. If construction fails, preserve the original list.

With a valid known predecessor and constant-time allocation, the update costs O(1). Finding the ith predecessor first costs O(i).

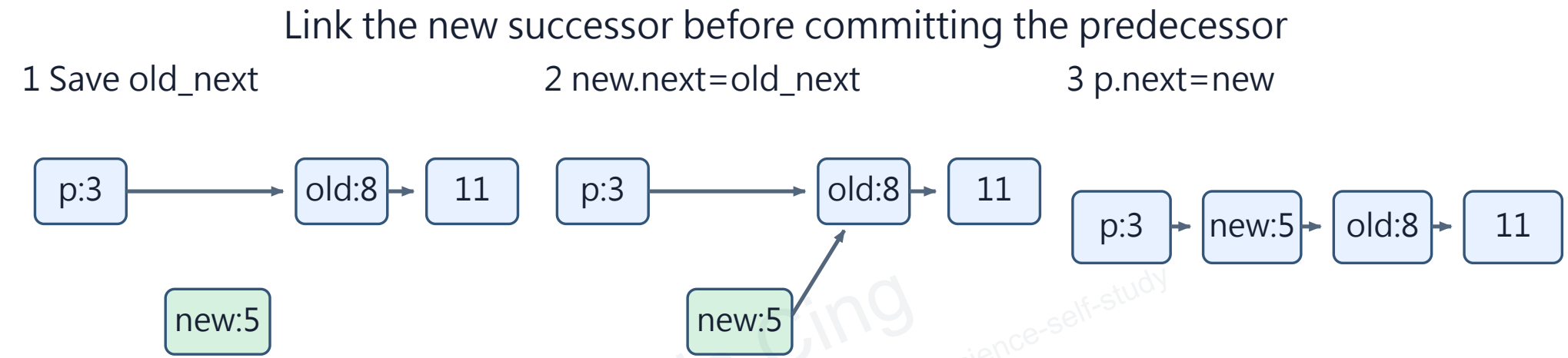
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete singly linked insertion and erasure

```
insert_after(p,value):
    require p is a live node of this container
    old_next=p.next
    x=allocate_and_construct(value)
    x.next=old_next
    p.next=x
    if tail==p: tail=x
    size+=1
erase_after(p):
    require p is live and p.next!=null
    victim=p.next; after=victim.next
    p.next=after
    if tail==victim: tail=p
    destroy_and_release(victim); size-=1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete singly linked insertion and erasure (continued)



Insert 5 after p=3 with successor chain 8→11. After allocation x is not reachable from head. Setting x.next=8 preserves the old 3→8→11 path and links x to 8. Committing p.next=x produces 3→5→8→11. Reversing the statements gives x.next=x, losing head reachability of 8 and 11. This is a structural reachability failure.

Complete singly linked insertion and erasure (continued)

Head updates need a separate case or a nondatum dummy predecessor. Expose an empty head/tail state consistently rather than treating the dummy as user data. With a known valid predecessor the link work is O(1); finding that predecessor by position remains O(n). Arbitrary external pointers do not satisfy the ownership precondition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reachability counterexample for reversed insertion assignments

```
old_next=p.next
p.next=x              // wrong first update
x.next=p.next         // reads x, not old_next
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reachability counterexample for reversed insertion assignments (continued)

Wrong order creates a self-loop and loses the old suffix



p.next=new; new.next=p.next

Unreachable from head

Starting at head=p in3→8→11, the wrong updates produce3→5→5→…. Nodes8,11 remain allocated but lose the owning path from the container. An external pointer retaining their address does not restore the representation. Correct insertion links x to saved old_next before committing p.next=x. The two writes are not automatically atomic for concurrent observers; this chapter assumes single-threaded structural updates.

Save the successor before deletion

- To delete after p, save `victim=p.next` and `after=victim.next`, set `p.next=after`, then release victim.
- If victim was the tail, set tail=p. Removing the only node requires both head and tail to become null.
- Dereference victim before release. Reading `victim.next` afterward is use-after-free.
- A pointer only to the node being removed does not generally permit O(1) deletion of a singly linked tail. Copying a successor's value also changes node-identity semantics.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Head operations, queue endpoints, and iterative clearing

```
push_front(v):
    x=Node(v,next=head); head=x
    if tail==null: tail=x
    size+=1
pop_front():
    require head!=null
    x=head; result=copy(x.value); head=x.next
    if head==null: tail=null
    release(x); size-=1; return result
enqueue(v):
    x=Node(v,next=null)
    if tail==null: head=x
    else: tail.next=x
    tail=x; size+=1
clear():
    while head!=null:
        x=head; head=x.next; release(x)
    tail=null; size=0
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Head operations, queue endpoints, and iterative clearing (continued)

These operations share one owning singly linked representation. A linked stack uses push_front/pop_front; a linked queue uses enqueue/pop_front. Assume single-threaded updates and successful allocation/value copying. Copy a possibly throwing result before changing links, so failure preserves the original list. Save the next pointer before releasing a node.

Starting at (head,tail,size)=(null,null,0), enqueue 4 makes both endpoints point to 4. Enqueue 9 gives 4→9 with size 2. Removing 4 leaves head=tail=9 and size 1; removing 9 clears both endpoints and size. Enqueue 7 then creates a valid new chain. Clearing head while retaining the released tail would make that last insertion write through a dangling pointer.

Inductively, insertion adds a fresh node through exactly one owning link, preserves acyclicity, and leaves tail.next=null. Removing the head preserves all remaining nodes and their order. Each update changes size with the reachable-node count. Endpoint operations take worst-case $O(1)$ under fixed-size value and constant allocation costs. Clearing releases each reachable node once in $\Theta(n)$ time and $O(1)$ extra stack space, avoiding a linear recursive-destructor call chain.

Three-pointer reversal invariant

Initialize `prev=null,curr=head`. Repeatedly save `next=curr.next`, set `curr.next=prev`, then advance prev and curr.

- Prev points to the reversed prefix and curr to the unprocessed suffix. Their node sets are disjoint and their union is the original list.
- After one iteration on `3→8→11`, the reversed part is `3→null` and the unprocessed part is `8→11`.
- At `curr=null`, set head=prev. If tail is maintained, the old head becomes the new tail.
- Every node is handled once. Time is Θ(n) and auxiliary pointer space is Θ(1).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete reversal and the disjoint-segment invariant

```
reverse(head):
    old_head=head; prev=null; curr=head
    while curr!=null:
        after=curr.next; curr.next=prev
        prev=curr; curr=after
    head=prev; tail=old_head
```

Reversing 3→8→11 gives (prev,curr) states (empty,3→8→11),(3,8→11),(8→3,11),(11→8→3,empty). Disjoint segments covering exactly the original node set prove preservation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C++ ownership and copying

- `unique_ptr` represents single ownership and transfers the release responsibility with ownership. A nonowning raw tail pointer still needs a validity invariant.
- Default shallow copying of owning raw pointers can make two containers release the same node. Define deep copying or explicitly prohibit copying.
- `shared_ptr` is not an automatic answer for every linked structure. Owning links in both directions or around a cycle create reference-count cycles.
- Recursive destruction of a long chain may create a deep call stack. A teaching implementation can clear iteratively and explain the lifetime behavior.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linked stacks and queues

Structure	Pointer arrangement	Update cost
Stack	Head is the top	O(1) insertion and deletion at head
Queue	Remove at head, insert at tail	O(1) each when tail is stored

- Enqueue into an empty linked queue makes both head and tail point to the new node.
- Dequeue of the last node must also set tail to null. Otherwise the next enqueue can write through a dangling pointer.
- These bounds exclude copying arbitrarily large values. An ADT does not guarantee actual allocator latency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Canonical polynomial representation

- A node stores `(exponent,coefficient)`. Exponents are nonnegative integers in strictly decreasing order, and coefficients are nonzero.
- `P=4x⁵−2x²+7` becomes `(5,4)→(2,-2)→(0,7)`.
- Duplicate exponents indicate unnormalized input. Combine their coefficients and remove a node when its coefficient cancels to zero.
- The empty list represents the zero polynomial. Do not mix it with a separate `(0,0)` representation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polynomial addition as sorted merging

- Let $Q = -4x^5 + 3x^3 + 2x^2$. Start two cursors at the largest exponents and compare current exponents.
- Output the term with the larger exponent and advance that cursor. For equal exponents, add coefficients, output only a nonzero sum, and advance both.
- The degree-five and degree-two terms cancel, giving $P + Q = 3x^3 + 7$.
- The output remains canonical and equals the sum of consumed input terms. Each node is advanced past once, giving $\Theta(p+q)$ time and $O(p+q)$ output space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polynomial merging with consumed-term states

```
add_polynomials(p,q):
    require both normalized, descending exponents
    out=[]
    while p!=end and q!=end:
        if p.exp>q.exp: append_copy(out,p); advance(p)
        else if q.exp>p.exp: append_copy(out,q); advance(q)
        else:
            c=p.coeff+q.coeff
            if c!=0: append(out,(p.exp,c))
            advance(p); advance(q)
    append_copy(out,remaining(p)); append_copy(out,remaining(q))
    return out
```

For $P=4x^5-2x^2+7$ and $Q=-4x^5+3x^3+2x^2$, equal exponents 5 cancel to zero. Comparing 2 and 3 emits $3x^3$. Equal exponents 2 cancel. Q ends, so append P's remaining 7. The result is $3x^3+7$. At least one cursor advances every iteration, and output remains descending with no zero coefficients.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polynomial merging with consumed-term states (continued)

Fixed integers require an overflow policy. A floating near-zero tolerance is not exact algebraic equality and changes the abstract value. Choose exact integers, rationals, or an explicitly approximate model before claiming polynomial identities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circular-list termination

- A nonempty circular singly linked list satisfies `tail.next=head`. It does not end at null. Stop traversal on returning to the starting node.
- With tail stored, head is `tail.next`. Inserting at either end changes a constant number of links.
- Starting at B in `A→B→C→A`, one traversal gives B, C, A. Waiting for null would never terminate.
- With a sentinel, an empty list may have the sentinel point to itself. A sentinel is a structural node, not user data.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circular tail insertion and endpoint costs

```
circular_push_back(value):
    x=new_node(value)
    if tail==null: x.next=x; tail=x
    else: x.next=tail.next; tail.next=x; tail=x
```

Circular insertion into empty gives A.next=A. Adding B gives tail=B,B.next=A,A.next=B. Traverse with a do/while stopping on return to head, checking tail=null first. With only tail stored, deleting the head is O(1), but deleting the tail needs predecessor search O(n).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Local consistency in doubly linked lists

- Nodes have prev and next. Neighboring nodes satisfy `x.next.prev=x` and `x.prev.next=x`, with endpoints handled by the sentinel policy.
- To insert x between adjacent p and q, set `x.prev=p,x.next=q,p.next=x,q.prev=x`.
- To delete known x, set `x.prev.next=x.next` and `x.next.prev=x.prev`, then release x.
- Updating a known node costs O(1). Finding a node by value still costs O(n). Removing a known node and searching then removing are different operations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Doubly linked erasure and reference lifetime

```
doubly_erase(x):
    require x is a live owned data node, x!=H and x!=T
    p=x.prev; q=x.next
    p.next=q; q.prev=p
    release(x); size-=1
```

This procedure fixes two nondata sentinels H and T. H.prev=null and T.next=null; an empty list has H.next=T, T.prev=H, and size=0. Every data node has nonnull prev/next, making p.next and q.prev valid. A null-ended representation instead needs explicit head/tail repair when erasing an endpoint and cannot directly use this procedure.

For one item H↔x↔T, p=H and q=T. Erasure restores H.next=T and T.prev=H and decreases size from 1 to 0. Never release H or T. The same sentinel links cover first/last data nodes in longer lists. Validate the live owned handle before entering pointer operations.

Deleting p↔x↔q sets both p.next=q and q.prev=p. The reciprocal conditions are restored together. References to surviving nodes remain valid; every reference to x is invalid. Updating only one direction gives inconsistent traversal sequences.

Choosing lists or arrays

Operation or constraint	Dynamic array	Singly linked list
Indexed access	O(1)	O(n)
Insertion with known predecessor	O(n) shifts in the worst case	O(1) link changes
Sequential scan	Usually storage locality	Pointer following
Extra space per item	Unused capacity	Next link and allocation overhead

The complete workload determines the tradeoff. Link counts alone do not establish measured speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sources and implementation boundaries

- The standard singly linked interface uses insert_after and erase_after. See C++ forward_list. For doubly linked costs see list overview.
- Ownership semantics appear in unique_ptr. Raw-pointer diagrams here are teaching models, not complete production containers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study